

Orchestration automatisée

Amazon ECS et Kubernetes

Binôme

Membre 1	Membre 2
Holali David GAVI	Claire Danièle EBA

Dépôt du projet

github.com/g-holali-david/automated-ecs-kubernetes-deployment

Année universitaire 2025 · 2026

Sommaire

1. Cadrage et architecture	3
2. Déploiement sur ECS	5
3. Déploiement sur Kubernetes.....	8
4. Automatisation avec Jenkins	13
5. Sécurité.....	14
6. Résilience.....	16
7. Reproductibilité et traçabilité.....	17
8. Difficultés rencontrées	18
9. Limites.....	19
10. Répartition du binôme	20
11. Bilan	21

1. Cadrage et architecture

Une même application conteneurisée doit tourner sur ECS pour la production cloud et sur Kubernetes pour la portabilité, les deux déploiements étant industrialisés par une seule chaîne d'automatisation.

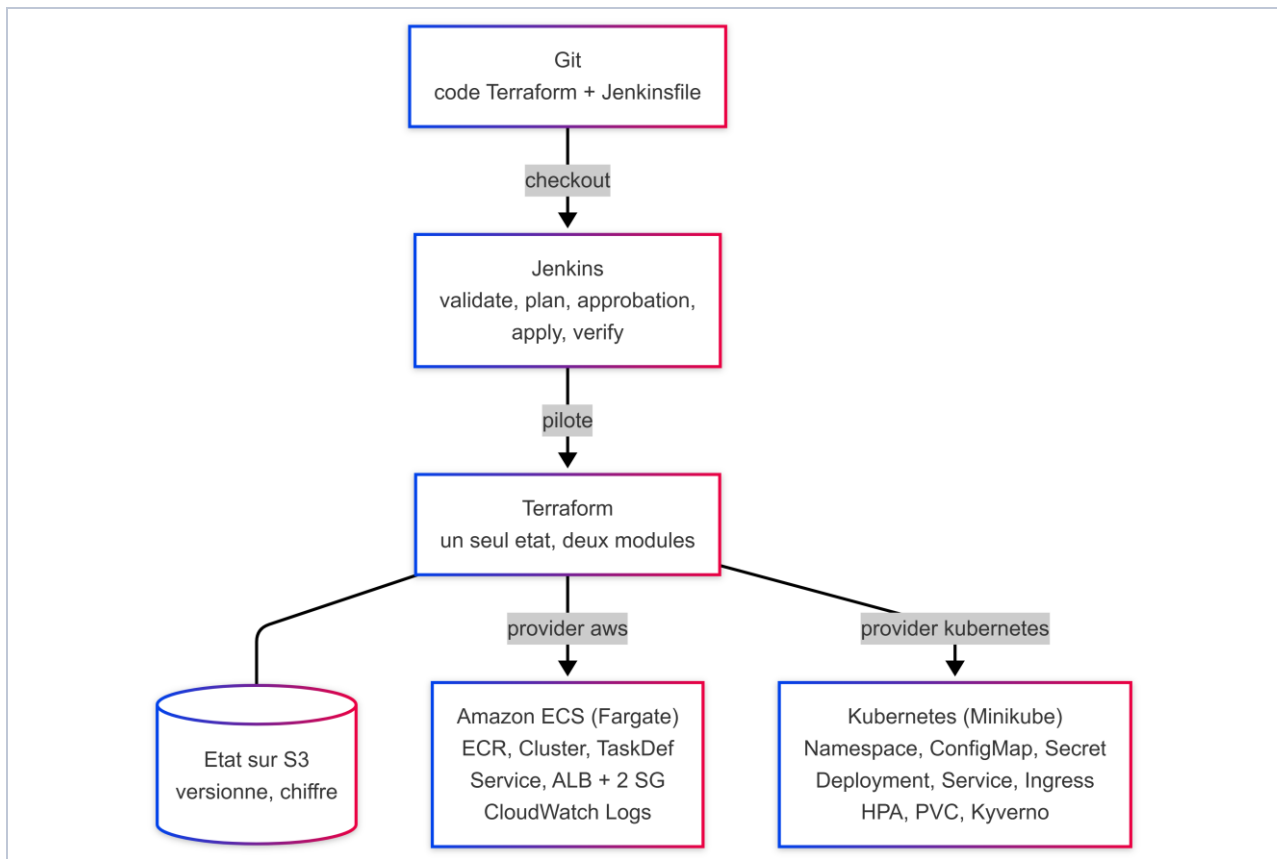


Schéma d'architecture : une seule chaîne Git, Jenkins, Terraform pilote les deux cibles depuis un état unique partagé sur S3.

Exigence	Traduction technique
Charge	2 tâches sur ECS, 3 réplicas sur Kubernetes, montée automatique jusqu'à 8
Disponibilité	Aucune coupure en mise à jour, redémarrage automatique d'une instance en échec
Sécurité	Moindre privilège réseau, aucun secret dans le dépôt, images à tag figé
Coût	Fargate en 0,25 vCPU et 512 Mio, rétention des logs à 7 jours

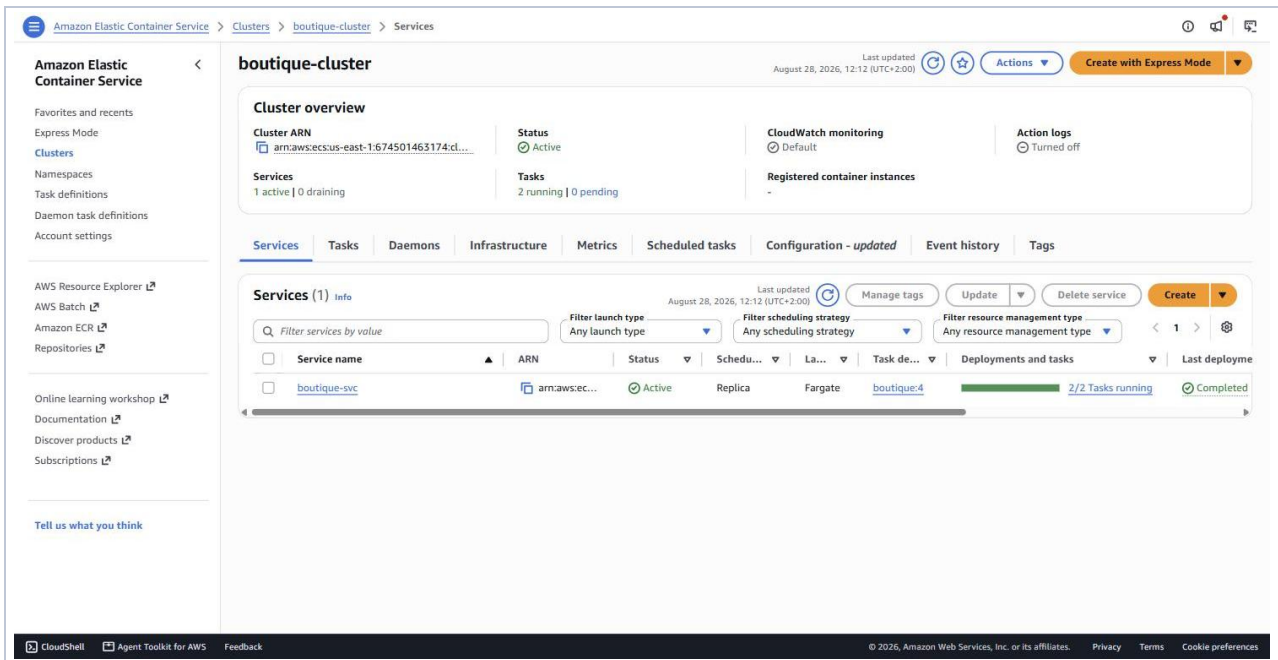
L'application est celle des TP précédents, en Node.js avec Express. Elle affiche son contexte d'exécution, ce qui permet de voir sur quelle cible on se trouve. Elle intègre une base PostgreSQL non demandée par le sujet, ajoutée pour aller plus loin, et démarre sans elle en mode dégradé.

AWS Academy impose `us-east-1`, interdit la création de rôles IAM et fournit des identifiants temporaires. `LabRole` sert donc d'exécution role. EKS étant indisponible pour la même raison, la cible Kubernetes est un cluster Minikube nommé `projet-ipssi`. Les ressources restent transposables vers EKS.

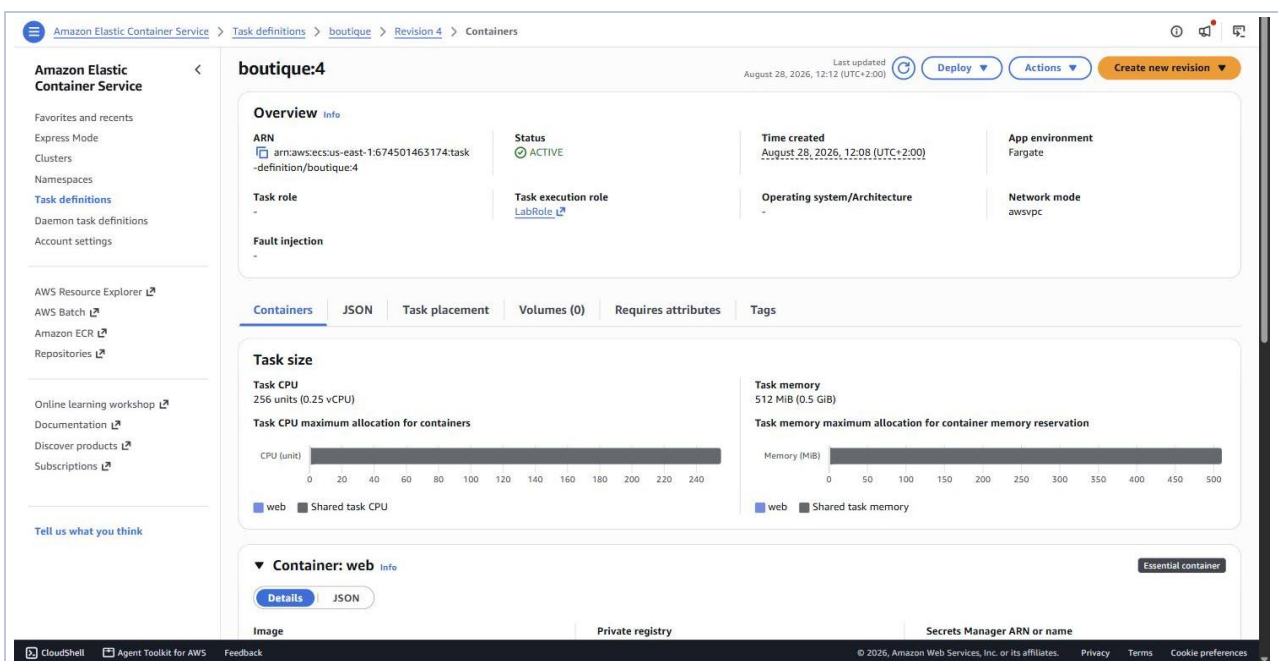
Il n'y a **qu'un seul état Terraform** pour les deux cibles : un `terraform apply` unique les réconcilie. Le code tient en deux modules, `modules/ecs` et `modules/k8s`, soit 26 ressources.

2. Déploiement sur ECS

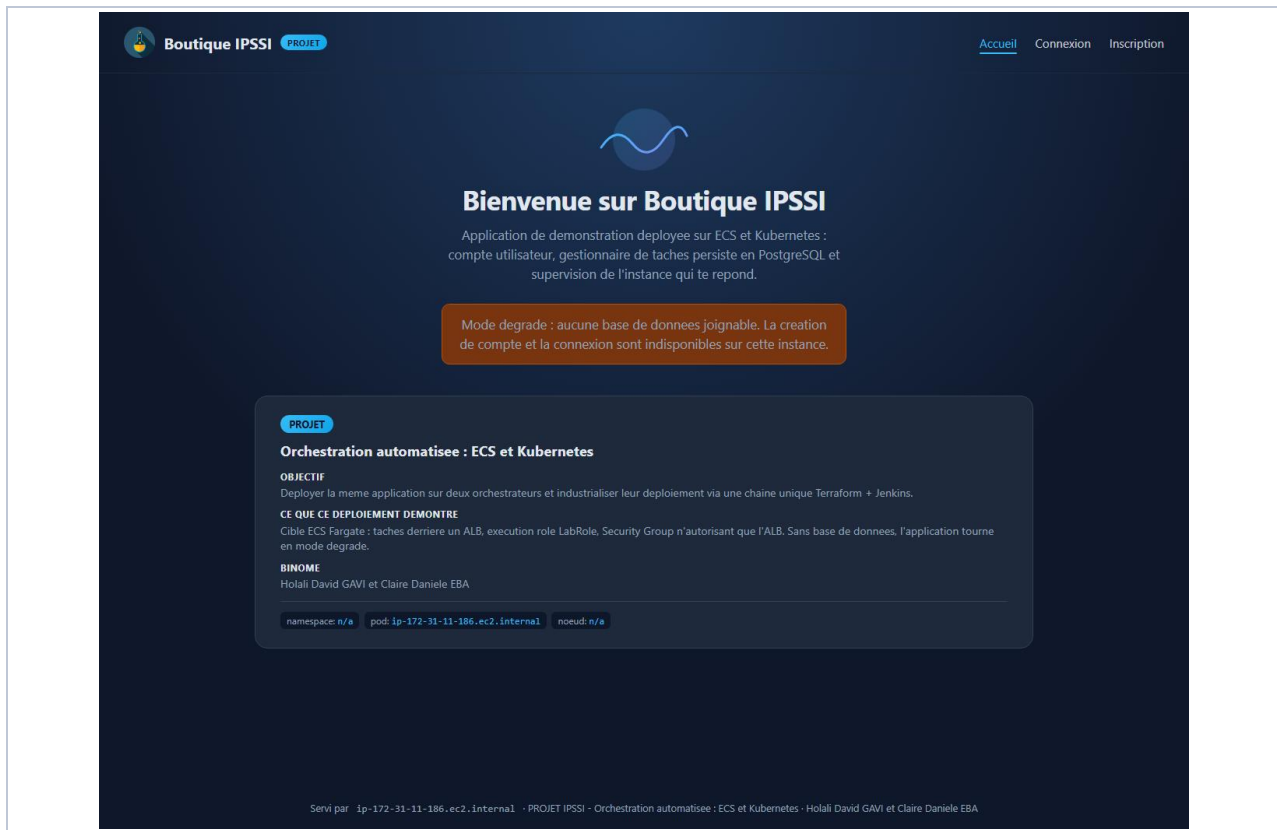
Le module `ecs` crée le dépôt ECR, le cluster Fargate, la task definition, le service, un ALB avec son target group et son listener, deux Security Groups et un groupe de logs CloudWatch. Le VPC et les sous-réseaux par défaut sont récupérés par des data sources, AWS Academy interdisant la création d'un réseau complet. Le target group est en mode `ip`, seule valeur possible avec le mode réseau `awsvpc` de Fargate.



Capture 02 : le service `boutique-svc`, 2 tâches sur 2 actives, déploiement terminé.

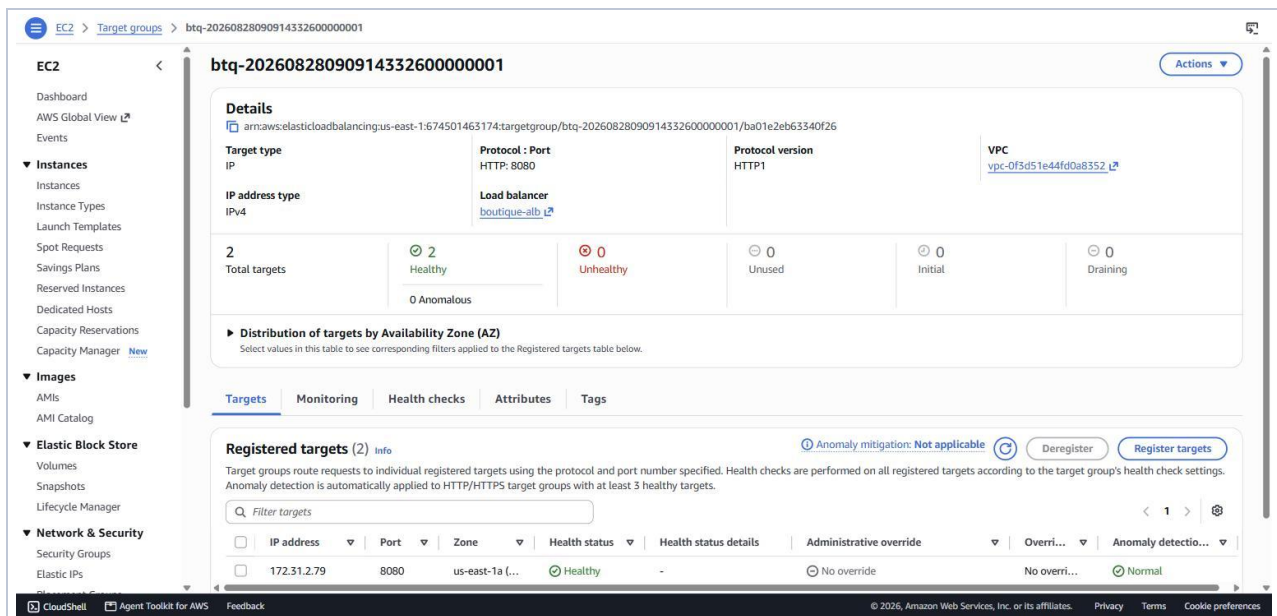


Capture 03 : execution role `LabRole`, mode réseau `awsvpc`, 0,25 vCPU et 512 Mio.

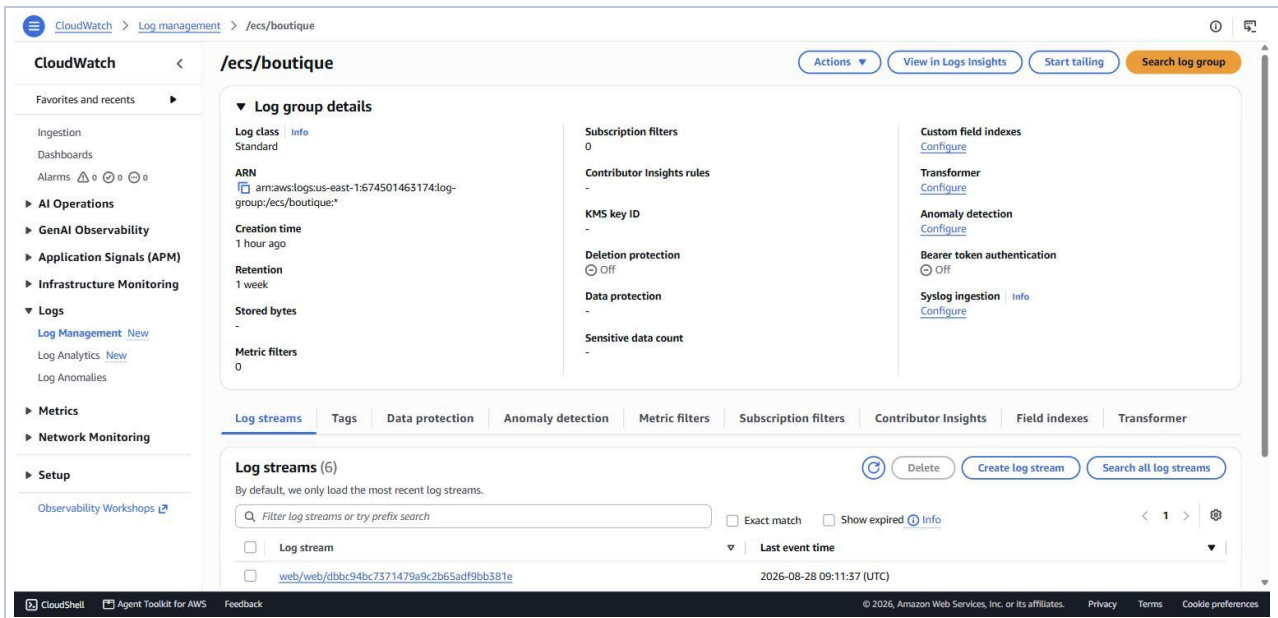


Capture 01 : l'application répond sur l'URL publique de l'ALB.

Le nombre de tâches est piloté par `ecs_desired_count`, fixée à 2. Le scheduler ECS relance automatiquement toute tâche arrêtée.



Capture 04 : les 2 cibles enregistrées sont en healthy sur le port 8080.



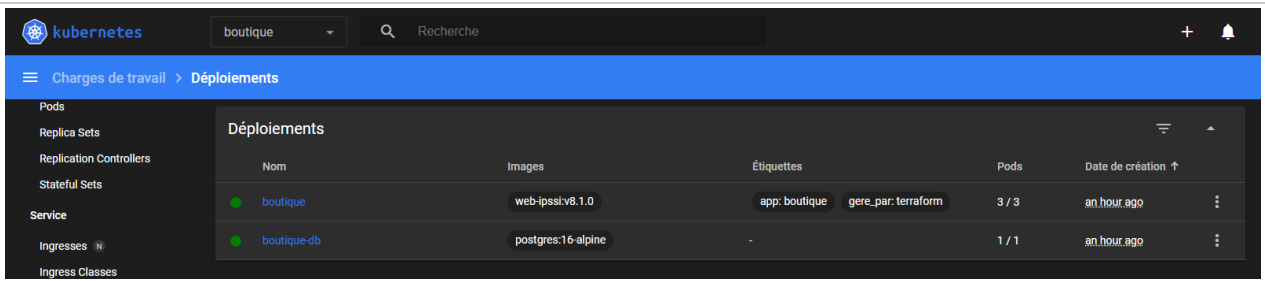
Capture 08 : le groupe de logs /ecs/boutique, un flux par tâche, rétention de 7 jours.

Pas de base de données sur cette cible. Le sujet n'en demande pas. Nous en avons déployé une uniquement sur Kubernetes. Sur ECS, RDS aurait dépassé le périmètre et un conteneur PostgreSQL dans la même tâche aurait perdu ses données à chaque redémarrage. C'est **le même artefact** sur les deux cibles, même image et même tag : seule la configuration d'exécution diffère, ce qui est le rôle de la ConfigMap d'un côté et des variables de la task definition de l'autre.

Sans base, l'application démarre en mode dégradé. Les sondes sont séparées en conséquence : la liveness interroge `/health`, qui répond toujours 200 tant que le processus tourne, la readiness interroge `/ready`, qui répond 503 sans base. Si la liveness avait dépendu de la base, l'application aurait redémarré en boucle alors qu'elle servait encore des pages.

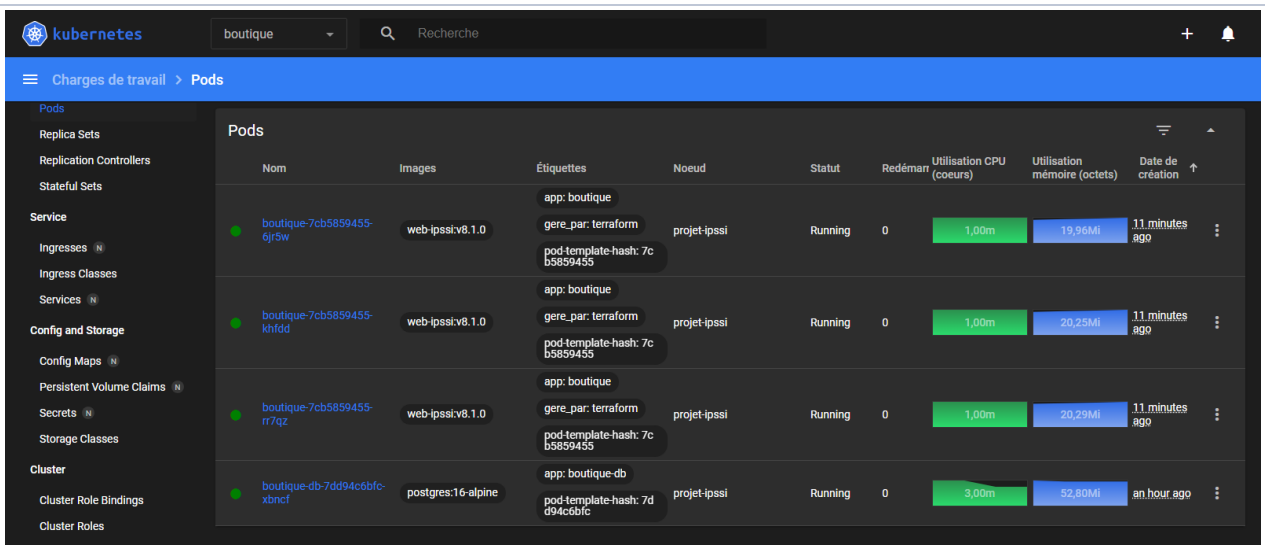
3. Déploiement sur Kubernetes

Le module `k8s` crée le namespace, une ConfigMap, un Secret, le Deployment de l'application, son Service en `ClusterIP`, l'Ingress, le HPA, la ClusterPolicy Kyverno, ainsi que le Deployment, le Service et la PVC de PostgreSQL. Aucun Service n'est exposé directement : l'entrée passe uniquement par l'Ingress.



Charges de travail > Déploiements						
	Nom	Images	Étiquettes	Pods	Date de création ↑	
Pods	boutique	web-ipssi:v8.1.0	app: boutique gere_par: terraform	3 / 3	an hour ago	
Replica Sets	boutique-db	postgres:16-alpine	-	1 / 1	an hour ago	

Capture 10 : les deux Deployments et leurs images à tag explicite.



Charges de travail > Pods									
	Nom	Images	Étiquettes	Noeud	Statut	Redémarr	Utilisation CPU (coeurs)	Utilisation mémoire (octets)	Date de création ↑
Pods	boutique-7cb5859455-6jr5w	web-ipssi:v8.1.0	app: boutique gere_par: terraform pod-template-hash: 7c b5859455	projet-ipssi	Running	0	1,00m	19,96Mi	11 minutes ago
Replica Sets	boutique-7cb5859455-khfd	web-ipssi:v8.1.0	app: boutique gere_par: terraform pod-template-hash: 7c b5859455	projet-ipssi	Running	0	1,00m	20,25Mi	11 minutes ago
Replication Controllers	boutique-7cb5859455-r7qz	web-ipssi:v8.1.0	app: boutique gere_par: terraform pod-template-hash: 7c b5859455	projet-ipssi	Running	0	1,00m	20,29Mi	11 minutes ago
Stateful Sets	boutique-db-7dd94c6bfc-xbncf	postgres:16-alpine	app: boutique-db pod-template-hash: 7d 094c6bfc	projet-ipssi	Running	0	3,00m	52,80Mi	an hour ago

Capture 11 : les quatre pods en Running sur le nœud projet-ipssi.

kubernetes

boutique



Recherche





Service > Services

Workloads

Cron Jobs

Daemon Sets

Deployments

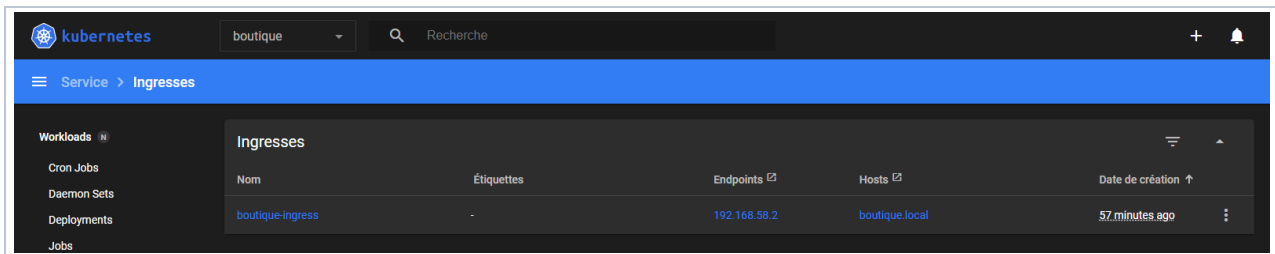
Jobs

Pods

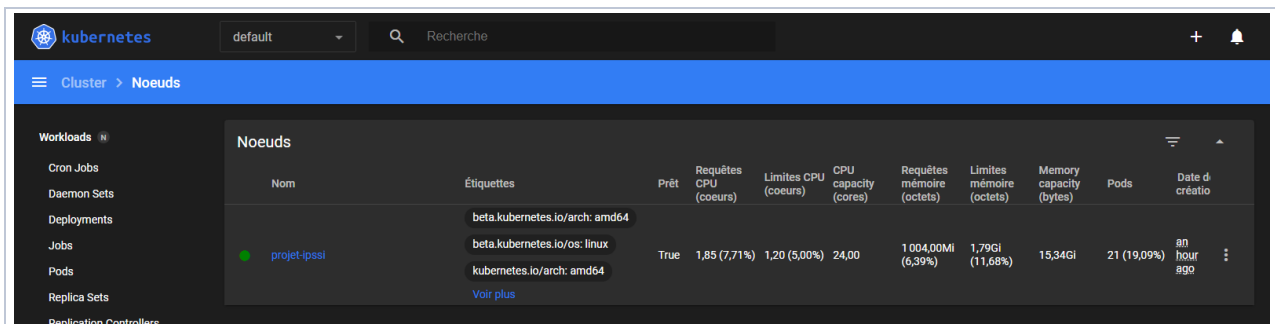
Services

Nom	Étiquettes	Type	IP cluster	Terminaisons internes	Terminaisons externes	Date de création
 boutique-db	-	ClusterIP	10.96.217.132	boutique-db.boutique:5432 TCP boutique-db.boutique:0 TCP	-	57 minutes ago
 boutique-svc	-	ClusterIP	10.97.171.200	boutique-svc.boutique:80 TCP boutique-svc.boutique:0 TCP	-	57 minutes ago

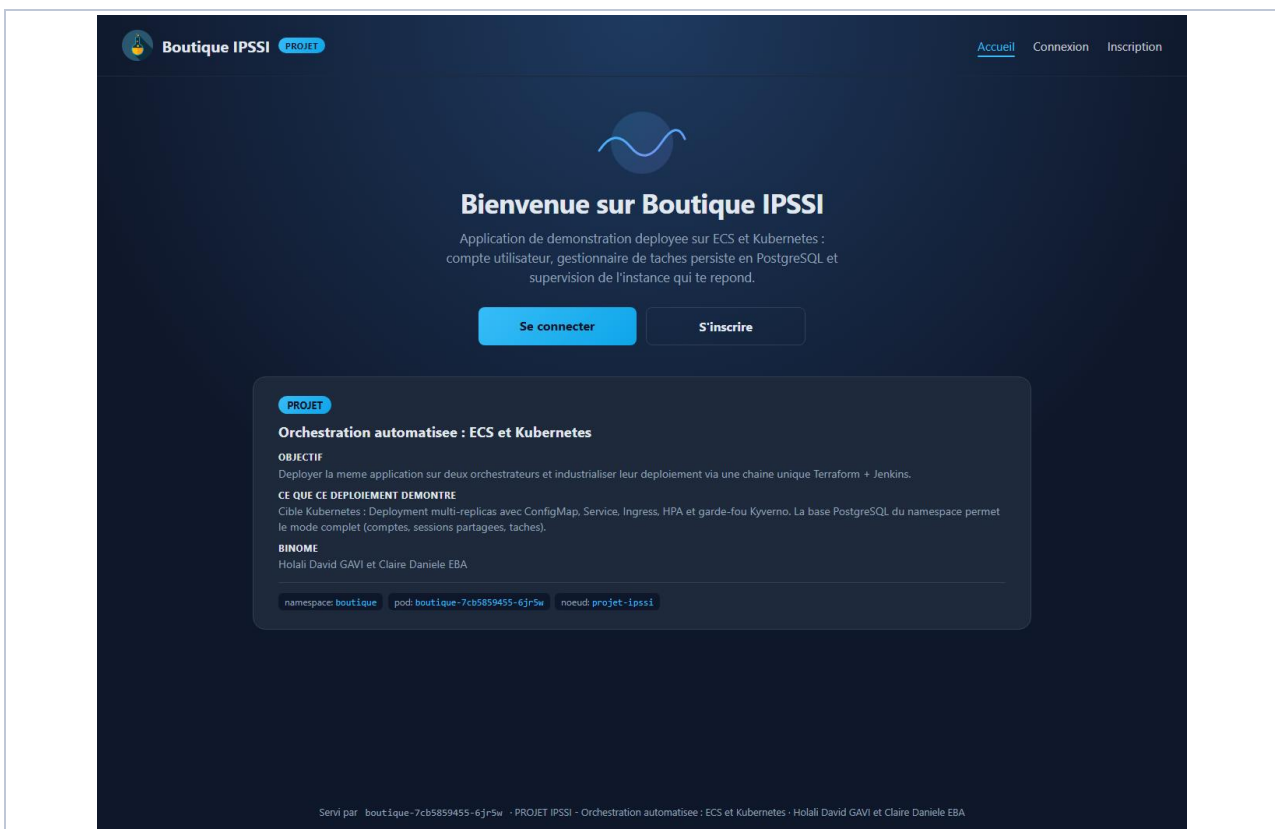
Capture 12 : les Services, tous deux en ClusterIP.



Capture 13 : l'Ingress boutique-ingress sur l'hôte boutique.local.

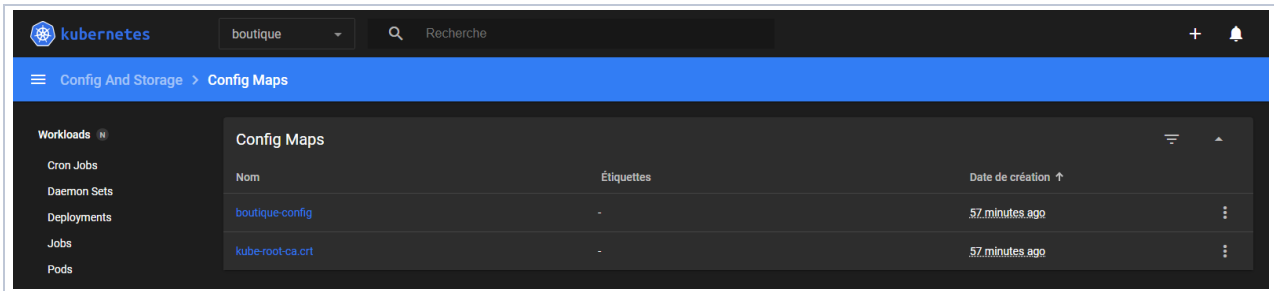


Capture 17 : le nœud projet-ipssi, nommé pour être distingué des clusters des TP précédents.

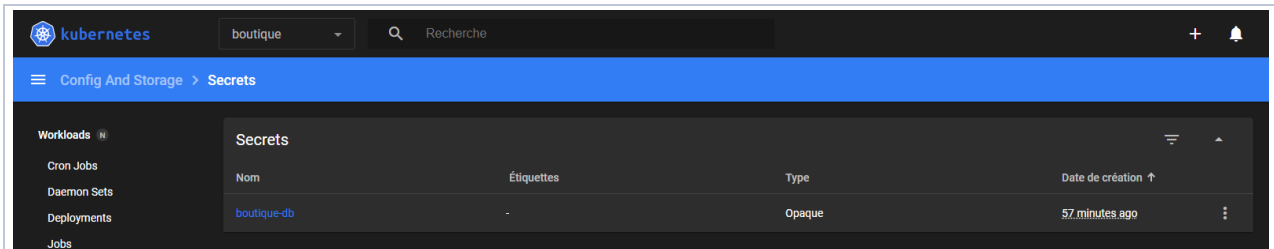


Capture 09 : l'application servie par l'Ingress. Le namespace, le pod et le nœud sont renseignés, contrairement à la capture 01 sur ECS.

Configuration et secrets. La configuration non sensible passe par une ConfigMap. Le mot de passe PostgreSQL est généré par Terraform avec `random_password` et stocké dans un Secret. Le pod reçoit aussi son nom, son namespace et son nœud par l'API Downward.

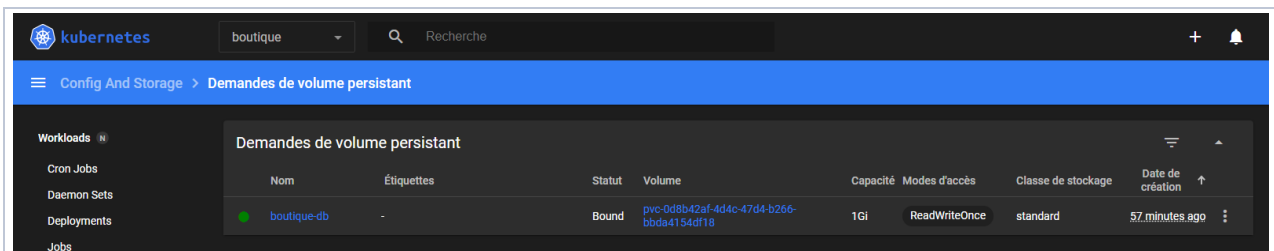


Capture 14 : la ConfigMap. Elle porte les coordonnées de la base, jamais le mot de passe.



Capture 15 : le Secret. Le mot de passe n'apparaît ni dans la ConfigMap ni dans le dépôt.

Persistence. PostgreSQL monte une PVC de 1 Gio en `ReadWriteOnce`, provisionnée dynamiquement. Sa stratégie est en `Recreate` et non `RollingUpdate`, un volume `ReadWriteOnce` n'étant montable que par un pod à la fois. Les données sont écrites dans un sous-répertoire du point de montage, celui-ci contenant un `lost+found` que l'initialisation de PostgreSQL refuse.



Capture 16 : la PVC est en Bound, liée à un PersistentVolume créé automatiquement.

Mise à l'échelle. Le HPA ajuste les réplicas entre 3 et 8 selon le CPU, cible à 50 %. Le calcul se fait sur les `requests` du Deployment, ici 50 millicores : sans elles, le HPA n'a pas de base de calcul. La fenêtre de stabilisation de descente est réduite de 300 à 30 secondes pour rendre la redescente observable en démonstration.

Horizontal Pod Autoscalers

Name	Namespace	Metrics	Min Pods
boutique-hpa	boutique	2% / 50%	3

HorizontalPodAutoscaler: boutique-hpa

Created: 73m 57s ago (2026-08-28T11:19:27+02:00)

Name: boutique-hpa

Namespace: boutique

Managed Fields: HashiCorp: Update, kube-controller-manager: Update

Reference: Deployment/boutique

Min Pods: 3

Max Pods: 8

Replicas: 3

Conditions: ScalingActive ScalingLimited AbleToScale

Metrics

Name	Current / Target
Resource cpu on Pods	2% / 50%

Events

Capture 18 : le HPA, ses seuils et ses conditions. `AbleToScale` confirme qu'il reçoit les métriques.

Garde-fou. Une ClusterPolicy Kyverno, décrite en Terraform, interdit en mode `Enforce` tout pod dont l'image porte le tag `latest`.

Workloads

Cron Jobs

Daemon Sets

Deployments

Jobs

Pods

Replica Sets

Replication Controllers

Stateful Sets

Service

Ingresses

Ingress Classes

Services

Config and Storage

Config Maps

Persistent Volume Claims

Secrets

Storage Classes

Cluster

Cluster Role Bindings

Cluster Roles

Events

Métadonnées

Nom	Date de création	Âge	UID
boutique-interdire-latest	28 août 2026	an hour ago	afb89167-6c26-46e7-b45c-c58f5c7b436c

Données

```

1 apiVersion: kyverno.io/v1
2 kind: ClusterPolicy
3 metadata:
4   creationTimestamp: '2026-08-28T09:19:02Z'
5   generation: 1
6   managedFields:
7     - apiVersion: kyverno.io/v1
8       fieldtype: FieldsV1
9       fieldsV1:
10        f:spec:
11          f:rules: {}
12        manager: Terraform
13        operation: Apply
14        time: '2026-08-28T09:19:02Z'
15        apiVersion: kyverno.io/v1
16        fieldtype: FieldsV1
17        fieldsV1:
18          f:status:
19            .: {}
20            f:autogen:
21              .: {}

```

Événements

Éléments : 0

Capture 19 : la ClusterPolicy. Le YAML indique `manager: Terraform`, donc elle vient du code et non d'un `kubectl apply` manuel.

```
kubectl run test-latest --image=nginx:latest -n boutique
```

```
holal@dev MINGW64 ~  
$ kubectl run test-latest --image=nginx:latest -n boutique  
Error from server: admission webhook "validate.kyverno.svc-fail" denied the request:  
  
resource Pod/boutique/test-latest was blocked due to the following policies  
  
boutique-interdire-latest:  
  verifier-tag-image: 'validation error: Un tag d''image explicite est requis (le tag latest est interdit).'  
  
holal@dev MINGW64 ~  
$ |
```

Capture 20 : le webhook refuse la création, avec le message écrit dans notre règle. Le pod n'est pas créé.

4. Automatisation avec Jenkins

Un `Jenkinsfile` unique, versionné, pilote les deux cibles. Le job est de type Pipeline from SCM : il lit le `Jenkinsfile` depuis Git, donc le pipeline évolue avec le code.

Étape	Rôle
Checkout	recupère le code versionné
Init	<code>terraform init</code> , connexion au backend S3
Validate	<code>terraform fmt -check -recursive</code> puis <code>terraform validate</code>
Plan	un plan couvrant les deux cibles, archivé comme artefact
Approve	le pipeline s'arrête et attend une validation humaine
Apply	applique le plan validé
Verify	interroge ECS et Kubernetes en parallèle

[Capture à insérer : 21-jenkins-pipeline.png]

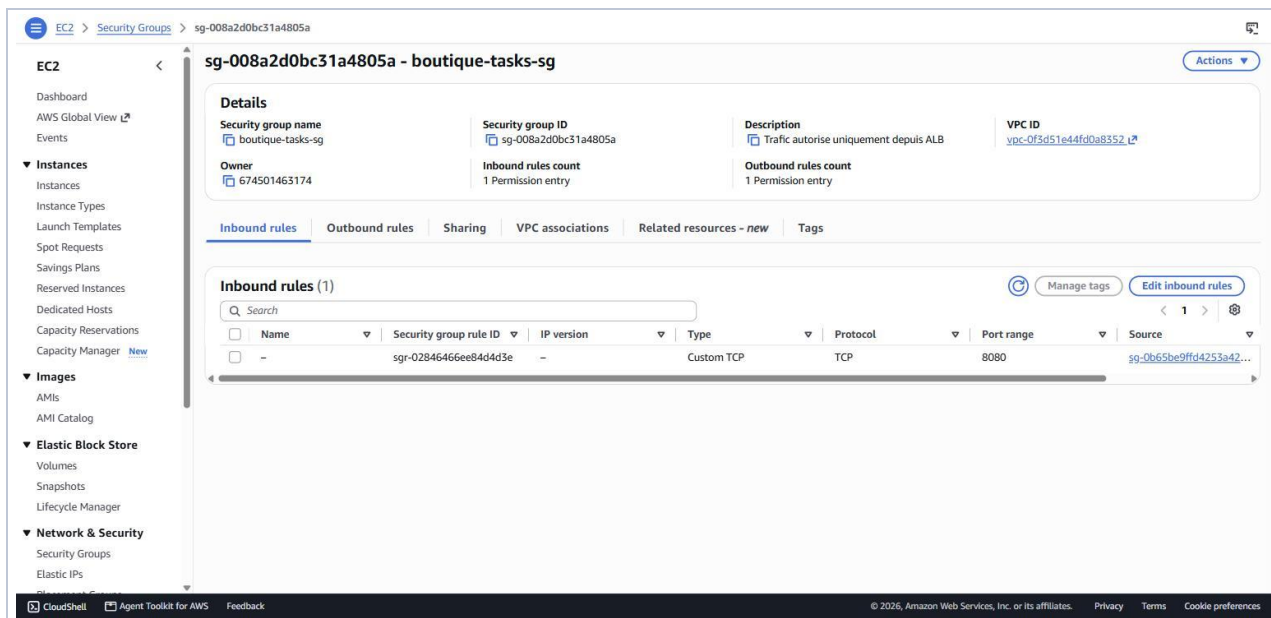
Capture 21 : le pipeline complet.

Idempotence. Sur une infrastructure déjà conforme, le plan annonce `No changes` et l'apply se termine par `0 added, 0 changed, 0 destroyed`. C'est le backend S3 qui le permet : Jenkins travaille dans un espace vierge à chaque exécution et, avec un état local, aurait tenté de recréer les 26 ressources existantes. Le bucket est versionné, chiffré en AES-256, accès public bloqué.

Gouvernance. L'étape `Approve` arrête le pipeline et affiche le plan avant application. Aucune modification ne part sans relecture humaine.

5. Sécurité

Moindre privilège réseau. Le Security Group de l'ALB accepte le HTTP depuis Internet, celui des tâches n'accepte que le port 8080 et uniquement depuis le Security Group de l'ALB. Une tâche n'est joignable qu'à travers le répartiteur de charge.

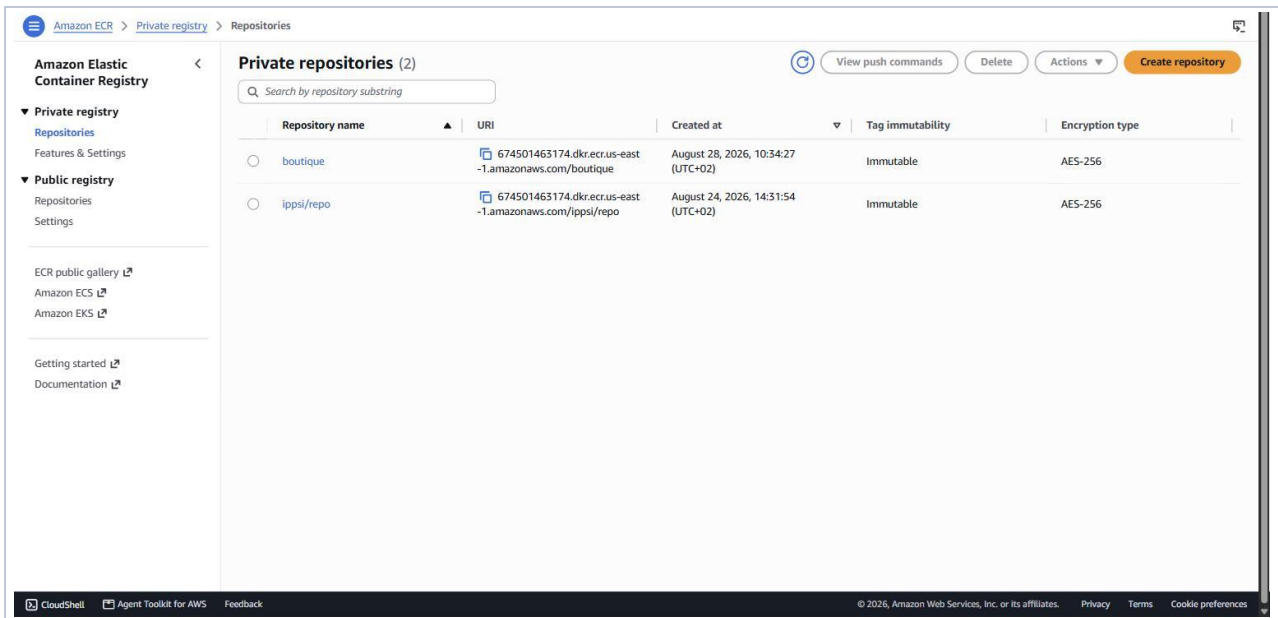


Capture 05 : une seule règle entrante, dont la source est le Security Group de l'ALB.

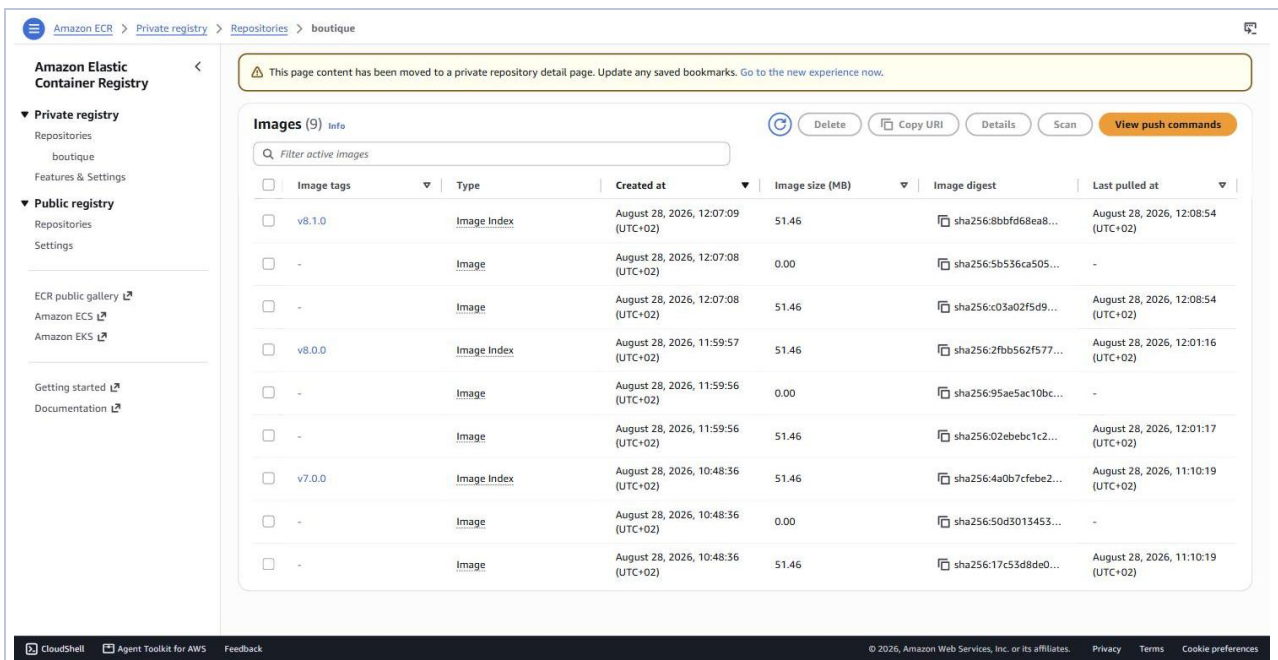
Moindre privilège IAM. L'exécution role est `LabRole`. Aucun rôle n'est créé par le projet.

Aucun secret en dur. Les identifiants AWS sont temporaires et jamais versionnés. `terraform.tfvars` et l'état Terraform sont exclus par `.gitignore`, l'état contenant en clair les valeurs des ressources. Le mot de passe PostgreSQL vit dans un Secret. Côté Jenkins, l'ARN du `LabRole` vient d'une variable d'environnement définie hors du dépôt.

Images taguées. Le tag `latest` n'est utilisé nulle part. Le dépôt ECR est en mode `IMMUTABLE`, donc un tag publié ne peut plus être écrasé, avec analyse de vulnérabilités à chaque push. Côté Kubernetes, Kyverno refuse tout pod sans tag explicite.



Capture 06 : le dépôt ECR en mode immuable, chiffré en AES-256.



Capture 07 : les versions successives coexistent avec des empreintes distinctes. Aucune image n'a été écrasée d'un déploiement à l'autre.

6. Résilience

	ECS	Kubernetes
Instances	2 tâches	3 réplicas
Auto-réparation	scheduler ECS	ReplicaSet
Sondes	health check de l'ALB	readiness et liveness
Zéro coupure	<code>minimum_healthy_percent = 100</code>	<code>maxUnavailable: 0</code> et hook <code>preStop</code>
Retour arrière	deployment circuit breaker	<code>kubectl rollout undo</code>
Mise à l'échelle	<code>desired_count</code>	HPA sur le CPU, de 3 à 8

Sur ECS, le circuit breaker annule et restaure automatiquement un déploiement qui échoue. Avec `minimum_healthy_percent` à 100, une nouvelle tâche est démarrée et déclarée saine avant qu'une ancienne soit retirée. Nous l'avons observé lors d'un changement de version : deux tâches sont restées en service pendant toute la bascule et l'ALB a répondu 200 sans interruption.

Sur Kubernetes, même principe avec `maxUnavailable: 0` et `maxSurge: 1`. Un hook `preStop` fait patienter le conteneur cinq secondes, le temps que le Service retire le pod de ses endpoints, ce qui évite de perdre les requêtes en cours.

7. Reproductibilité et traçabilité

Les deux déploiements se recréent intégralement depuis le dépôt. Trois éléments seulement dépendent du poste et sont documentés dans le README : les identifiants AWS de la session Academy, le nom du bucket d'état, et le chargement de l'image dans Minikube, ce dernier point venant d'une limite d'Academy puisque Minikube ne peut pas s'authentifier auprès d'ECR sans imagePullSecret. Le projet se redéploie sur un autre compte Academy sans modifier une ligne de code :

```
terraform init -reconfigure -backend-config="bucket=tfstate-ipssi-<ID_DU_COMPTE>"
```

L'historique Git est fait de commits atomiques aux messages explicites. L'état sur S3 est versionné, ce qui permet de savoir quand une ressource a changé et de revenir en arrière. Le plan de chaque exécution Jenkins est archivé comme artefact du build.

8. Difficultés rencontrées

Problème	Cause	Solution
Suppression du target group refusée	le listener le référence encore	<code>name_prefix</code> et <code>create_before_destroy</code>
CrashLoopBackOff au premier déploiement	port 80 déclaré au lieu de 8080	correction dans les deux modules
PostgreSQL refuse de s'initialiser	<code>lost+found</code> dans le point de montage	données dans un sous-répertoire
HPA sans effet	fenêtre de stabilisation trop longue	<code>select_policy</code> et fenêtre à 30 s
Jenkins voulait recréer les 26 ressources	espace de travail vierge, état local	backend S3 partagé
Vérification ECS en échec dans Jenkins	l'AWS CLI reprenait la région du poste	<code>AWS_DEFAULT_REGION</code> figée dans le Jenkinsfile

La dernière est instructive : l'erreur mentionnait une ressource en `eu-west-3` alors que le projet est en `us-east-1`. Le fichier de configuration de l'AWS CLI du poste portait encore cette région. Terraform n'était pas affecté, son provider fixant la région explicitement, mais l'AWS CLI appelée dans l'étape de vérification, si.

9. Limites

La cible Kubernetes est un Minikube local. Vers EKS, l’Ingress devrait être remplacé par un contrôleur adossé à un load balancer AWS et le stockage par une StorageClass EBS.

Le pipeline s’arrête au déploiement de l’infrastructure : la construction et la publication de l’image restent manuelles. Enfin, les identifiants AWS sont copiés vers le compte de service Jenkins par un script ; en production on utiliserait le gestionnaire d’identifiants de Jenkins ou un rôle IAM porté par l’agent.

10. Répartition du binôme

Chacun a écrit sa part du code, puis déployé et exploité sa propre stack, sur son compte AWS Academy et son cluster Minikube. Les deux membres savent expliquer l'ensemble de la chaîne.

Tâche	Holali David GAVI	Claire Danièle EBA
Cadrage du besoin	Rédaction	Relecture
Schéma d'architecture	Relecture	Rédaction
Module Terraform <code>ecs</code>	Relecture	Rédaction
Module Terraform <code>k8s</code>	Rédaction	Relecture
Scripts d'exploitation AWS		Rédaction
Scripts d'exploitation Kubernetes	Rédaction	
Pipeline Jenkins	Version écrite puis comparée	Version écrite puis comparée
Rapport écrit	Rédaction	Relecture
Déploiement, captures, démonstration	Sa propre stack	Sa propre stack

Chacun a écrit son propre `Jenkinsfile`. Nous les avons comparés puis retenu une seule version pour le dépôt commun. Les captures de chaque stack sont dans le dépôt, sous `docs/captures-stack-david` et `docs/captures-stack-daniele`.

11. Bilan

Les quatre briques sont en place : la même application tourne sur ECS Fargate et sur Kubernetes, les deux déploiements sont décrits par un seul code Terraform en deux modules, et un pipeline Jenkins gouverné les applique après approbation.

Ce que le projet nous a le plus appris n'est pas la syntaxe de Terraform mais la gestion de l'état. Tant qu'il restait local, l'automatisation n'était qu'une apparence : le pipeline aurait détruit et recréé une infrastructure existante. Le déplacer sur S3 a rendu la chaîne réellement idempotente.